

Resenha crítica

On the Criteria To Be Used in Decomposing Systems into Modules (D. L. Parnas, 1972)

O texto de Parnas é frequentemente tratado como clássico, mas o valor dele hoje não pode ser aceito só por tradição histórica. Ele continua importante porque ataca um problema que ainda define custo e qualidade de software moderno: como modularizar sem transformar cada mudança em efeito dominó. Em ambientes atuais, isso aparece de forma concreta em arquiteturas de microserviços, em pipelines de CI/CD e em times que fazem deploy contínuo. Quando as fronteiras entre módulos são mal escolhidas, qualquer alteração pequena exige coordenação entre vários serviços, quebra contratos de API e aumenta o tempo de entrega. Nesse sentido, o artigo continua relevante não por ser “antigo e famoso”, mas porque descreve uma dor que hoje é até mais cara do que em 1972.

Parnas constrói sua tese usando o sistema KWIC, comparando uma decomposição tradicional, orientada por etapas de processamento, com outra baseada em ocultação de decisões de projeto (information hiding). O exemplo é didático e funciona para mostrar a diferença entre “dividir por fluxo” e “dividir por volatilidade de decisão”. Ao mesmo tempo, essa escolha metodológica é um ponto fraco: KWIC é um problema pequeno, com baixa complexidade organizacional e técnica. Isso facilita provar a elegância da proposta, mas dificulta demonstrar escalabilidade para cenários com concorrência intensa, integração com sistemas legados, requisitos regulatórios ou equipes grandes e distribuídas. Em outras palavras, o exemplo convence no plano conceitual, mas não resolve a pergunta mais difícil: o princípio se mantém igualmente eficiente quando o sistema deixa de ser didático e vira produto crítico de mercado?

A ideia de information hiding é, sem dúvida, o núcleo intelectual mais forte do artigo. Parnas acerta ao defender que cada módulo deve encapsular decisões propensas a mudança, reduzindo acoplamento estrutural e risco de propagação de impacto. Esse princípio conversa diretamente com práticas atuais como desenho por contratos, separação entre domínio e infraestrutura e versionamento de interfaces. Porém, uma leitura realmente crítica exige reconhecer que ocultação de informação não é solução universal. Em sistemas de tempo real, software embarcado e aplicações de altíssima performance, abstrações adicionais podem introduzir overhead inaceitável de latência, consumo de memória ou custo de sincronização. Além disso, encapsulamento excessivo pode produzir o efeito contrário ao desejado: fronteiras “limpas” no papel, mas difíceis de depurar em produção, especialmente quando observabilidade e tracing distribuído são insuficientes. Assim, o princípio é poderoso, mas precisa de calibração contextual.

Um mérito importante do artigo é que Parnas não ignora trade-offs de eficiência. Ele admite que modularização mal implementada pode custar desempenho. Essa honestidade técnica fortalece o texto, mas hoje podemos aprofundar esse ponto com exemplos mais concretos: em arquiteturas de microserviços, a fragmentação exagerada cria custo de rede, serialização, retries e consistência eventual; em plataformas DevOps, limites mal definidos geram pipelines lentos e testes de integração frágeis; em sistemas orientados a eventos, a busca por isolamento pode aumentar complexidade operacional e dificultar diagnóstico de falhas. Portanto, o debate proposto por Parnas permanece válido, só que em escala ampliada: o problema já não é apenas “chamar muitas rotinas”, e sim administrar custo sistêmico de fronteiras arquiteturais.

Quando o artigo é comparado com outras filosofias de projeto, o ganho analítico fica mais claro. A programação estruturada tradicional tende a privilegiar decomposição por sequência lógica de processamento, o que ajuda legibilidade inicial, mas costuma espalhar decisões internas por vários pontos do sistema. A orientação a objetos, por outro lado, absorve parcialmente a proposta de Parnas ao promover encapsulamento e interfaces, embora na prática muitas bases OO acabem acopladas por herança excessiva ou objetos anêmicos sem real proteção de decisão. Já abordagens contemporâneas como Domain-Driven Design e arquitetura hexagonal podem ser vistas como extensões do raciocínio de Parnas: o foco deixa de ser “etapas do algoritmo” e passa a ser “fronteiras de responsabilidade e mudança”. Isso mostra que a contribuição dele não compete com todas as abordagens posteriores; em muitos casos, ela funciona como fundamento conceitual delas.

A principal limitação do artigo, portanto, não é teórica, mas de validação. Falta evidência empírica robusta para sustentar que a abordagem escala com previsibilidade em diferentes contextos industriais. Sem dados comparativos de produtividade, taxa de defeitos, custo de manutenção e impacto organizacional, parte da força do texto permanece no campo argumentativo. Isso não invalida sua contribuição, mas impede tratá-la como verdade fechada. Um trabalho fundacional pode ser intelectualmente brilhante e, ainda assim, metodologicamente incompleto para padrões atuais de evidência em engenharia.

No balanço final, Parnas oferece uma virada de mentalidade indispensável: modularizar não é só separar partes de execução, mas proteger decisões instáveis para preservar evolutividade. Essa ideia continua central para engenharia de software moderna. Ainda assim, o artigo não deve ser lido como manual definitivo, e sim como ponto de partida rigoroso que precisa ser complementado por contexto arquitetural, métricas empíricas e análise de custo operacional. A obra é fundamental, mas incompleta; influente, mas não autossuficiente. É justamente essa combinação que explica por que ela segue obrigatória em formação acadêmica séria: não porque encerra o debate, e sim porque inaugura o debate certo.